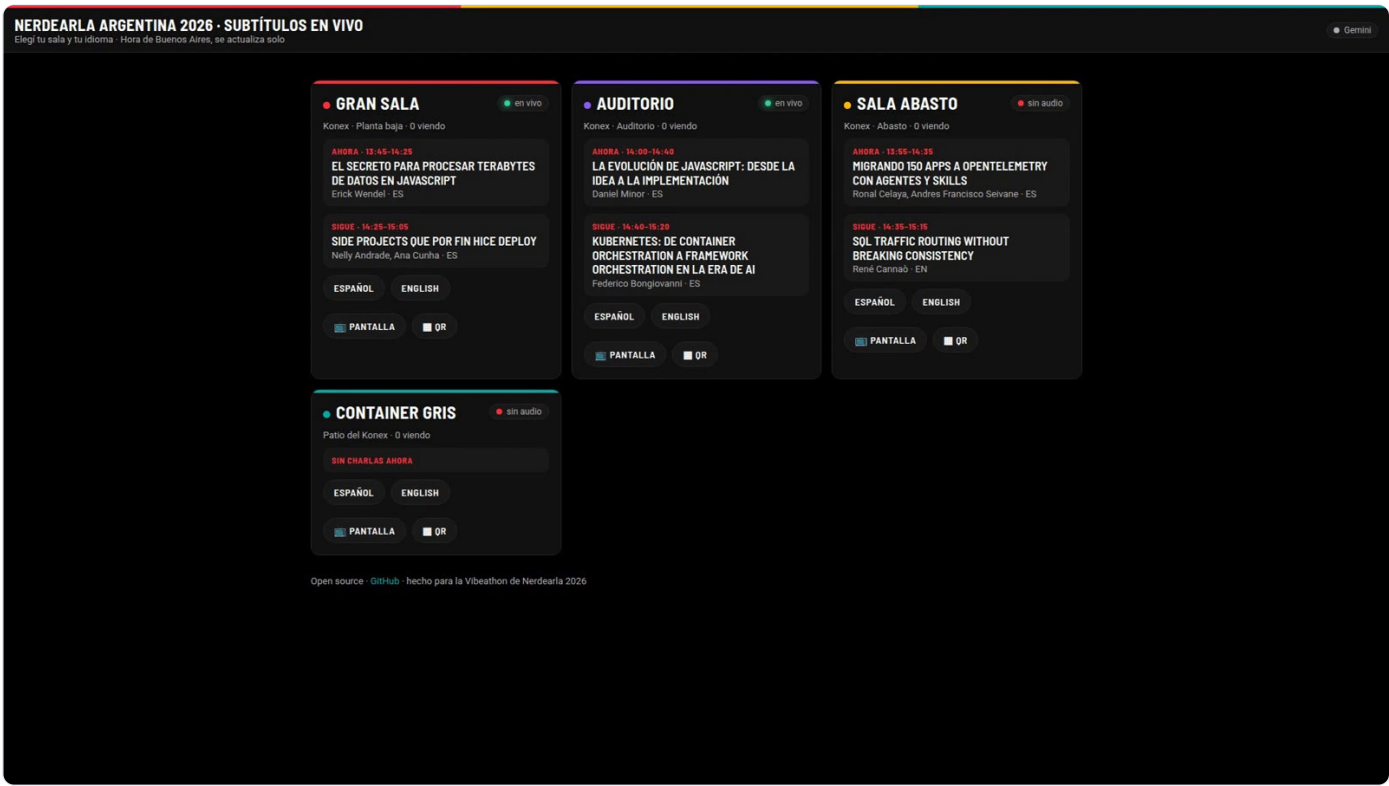


Guía para el equipo de producción de una conferencia. Cubre la instalación, la configuración de los escenarios, la operación durante el evento y qué hacer si algo falla.

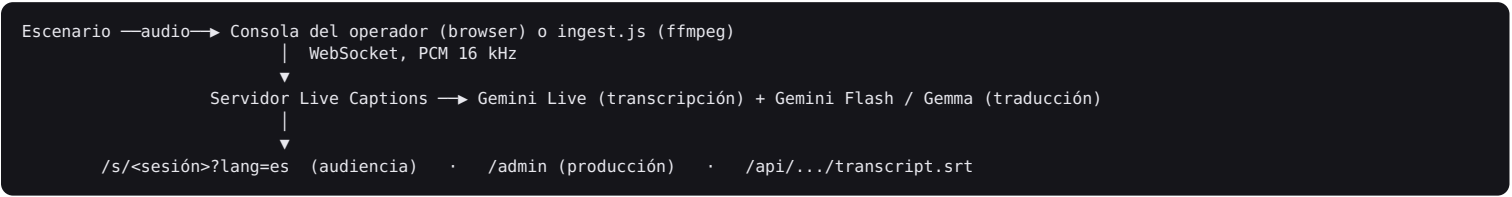
Todas las capturas de esta guía son de la demo pública corriendo en vivo (no mockups).



Portada: salas con agenda en vivo de Nerdearla 2026

1. Qué es y cómo funciona

Live Captions toma el audio en vivo de cada escenario y produce subtítulos en tiempo real en el idioma original y traducidos (español, inglés, portugués u otros). La audiencia los ve desde el celular o desde una pantalla de la sala, eligiendo sesión e idioma.



Tres roles:

- **Producción:** instala el servidor, define las sesiones, vigila `/admin`.
- **Operador de escenario:** abre `/operator/<sesión>` en la notebook de la sala y pone a transmitir el audio.
- **Audiencia:** abre `/` (o el QR de la sala), elige sesión e idioma.

2. Instalación (una vez, antes del evento)

Requisitos

- Un servidor Linux con Docker (1 vCPU y 1 GB de RAM alcanzan para ~10 sesiones) o Node.js 20+.
- Un dominio con HTTPS (por ejemplo `captions.miconferencia.org`). Es obligatorio si vas a usar la consola del operador en el navegador: los navegadores sólo dan acceso al micrófono en páginas seguras.
- Una API key de Gemini: `https://aistudio.google.com` → *Get API key*. Activá la facturación del proyecto de Google Cloud para no quedar limitado por las cuotas del nivel gratuito (15 pedidos por minuto por modelo).

En la nube en un clic

- **Render:** *New* → *Blueprint*, elegí el repo (usa `render.yaml`); cargá `GEMINI_API_KEY` y `PUBLIC_URL`. Así está montada la demo pública.
- **Railway:** *New project* → *Deploy from GitHub* (usa `railway.json`).

- **Google Cloud Run:** `GEMINI_API_KEY=... ./deploy/cloudrun.sh <proyecto> southamerica-east1`.
- **Kubernetes:** `kubectl apply -f deploy/kubernetes.yaml` (creá antes el secret `live-captions`).

Con Docker (recomendado)

```
git clone https://github.com/lucasmde/nerdearla-live-captions
cd nerdearla-live-captions
cp .env.example .env           # editar: GEMINI_API_KEY e INGEST_TOKEN
cp sessions.example.json sessions.json  # editar: tus escenarios
docker compose up -d --build
```

El servidor queda en el puerto 8080. Ponelo detrás de un reverse proxy con TLS. Con Caddy es una línea:

```
captions.miconferencia.org {
    reverse_proxy localhost:8080
}
```

Sin Docker

```
npm install
npm start
```

Variables de entorno importantes (`.env`)

Variable	Para qué sirve
GEMINI_API_KEY	Clave de Gemini. Sin ella el servidor arranca en modo demo (<code>ENGINE=mock</code>).
INGEST_TOKEN	Contraseña que deben conocer los operadores para enviar audio. Siempre poné una en producción.
TRANSCRIBE_MODEL	Modelo de transcripción en streaming (<code>gemini-3.5-transcribe-live</code>).
TRANSLATE_MODEL	Modelo de texto para traducir (<code>gemini-3.5-flash-lite</code>).
TRANSLATE_FALLBACK_MODELS	Modelos alternativos si el principal está saturado.
TRANSLATE_PARTIALS	<code>1</code> traduce también las frases a medio decir (menos espera, más costo). <code>0</code> sólo traduce frases completas.
TRANSLATOR	<code>gemini</code> (nube), <code>ollama</code> (Gemma local) o <code>none</code> .

3. Configurar los escenarios (`sessions.json`)

```
{
  "vocabulary": ["Nerdearla", "Kubernetes", "PostgreSQL", "Sysarmy"],
  "sessions": [
    {
      "id": "gran-sala", "name": "Gran sala", "room": "Konex", "color": "#FF323C", "sourceLang": "auto", "targetLangs": ["es", "en"],
      "agenda": [{ "day": "2026-09-25", "start": "13:45", "end": "14:25", "title": "El secreto para procesar terabytes de datos en JavaScript",
      "speaker": "Erick Wendel", "lang": "es" } ] },
    { "id": "sala-b", "name": "Sala B", "room": "Piso 2", "sourceLang": "en", "targetLangs": ["es", "en"] },
    { "id": "taller", "name": "Taller", "room": "Lab", "sourceLang": "es", "targetLangs": ["es", "en", "pt"] }
  ]
}
```

- `id` : corto, sin espacios; aparece en las URLs (`/s/main`).
- `sourceLang` : `auto` si en ese escenario hay charlas en varios idiomas; un código fijo (`en` , `es`) mejora la precisión cuando el idioma es siempre el mismo.
- `targetLangs` : los idiomas que la audiencia puede elegir. Incluí siempre el idioma original de las charlas para que exista la opción “ver la transcripción literal”.
- `color` : color de la sala (borde de la tarjeta y barra superior). `agenda` : charlas con `day` , `start` , `end` , `title` , `speaker` , `lang` ; el sistema muestra “ahora / sigue” en hora del evento (`timezone` en el archivo) y suma los oradores al vocabulario.
- `vocabulary` : nombres propios, siglas y términos técnicos del evento. Mejora mucho el reconocimiento de nombres raros. Se puede poner uno global y otro por sesión.

Después de editar `sessions.json` a mano, reiniciá el servidor (`docker compose restart`).

3.1. Panel de salas (`/admin/sessions`)

Para no tener que editar `sessions.json` a mano ni reiniciar el servidor, hay un panel web para crear salas y cargar la agenda en caliente:

1. `ADMIN_EMAILS=vos@gmail.com,otro-organizador@gmail.com` en `.env` (o en las variables de entorno del hosting) define quién administra **todas** las salas, incluidas las reales del evento. Vacío = nadie tiene ese nivel.
2. Necesita el login con Google o GitHub configurado (sección 8).
3. Entrá a `https://tu-dominio/admin/sessions` (hay un link “Salas y agenda” arriba del panel de producción `/admin`).
4. **No hace falta estar en `ADMIN_EMAILS` para probar el panel.** Cualquier cuenta de Google o GitHub que inicie sesión ahí puede crear su propia sala de prueba y armarla de punta a punta —agenda, código de orador, todo— sin pedirle nada a quien administra el evento. Cada cuenta ve y edita solo

- la o las salas que creó ella misma; nunca las de otra cuenta ni las salas reales del evento (esas quedan protegidas y solo las tocan los mails de `ADMIN_EMAILS`). Es la forma pensada para que un jurado, un compañero de equipo o cualquiera que quiera ver “cómo se arma un evento” lo pruebe sin depender de que el organizador esté disponible para darle el alta.
5. **Nueva sala:** id (se usa en la URL, ej. `sala-abasto`), nombre, ubicación, color, idioma de origen, a qué idiomas traducir, y una “fuente de audio prevista” — un campo de texto libre (link de Zoom, URL de streaming, `rtmp://...` para `tools/ingest.js`, o simplemente una nota como “pestaña del canal oficial”). Ese campo es solo informativo y queda visible para el operador en `/operator/<id>`; la captura real de audio (micrófono o pestaña) se sigue iniciando en vivo desde ahí, como siempre.
 6. Cada sala ya creada se puede editar (nombre, ubicación, colores, idiomas, fuente de audio) y tiene su propia tabla de agenda: agregá charlas con título, orador, horario de inicio/fin e idioma, y editá o borrá cada una con los botones de la fila. Los nombres de los oradores se suman automáticamente al vocabulario de la sala (mejora el reconocimiento de nombres propios).
 7. Una sala que está recibiendo audio en ese momento no se puede borrar (hay que detener la captura desde el operador primero); sí se puede seguir editando su nombre/agenda mientras está en vivo. Para que nadie deje el servidor lleno de salas de prueba, hay un tope de 60 salas en total y de 6 salas nuevas cada 10 minutos por IP.

Los cambios se guardan en un `sessions.json` que vive en `DATA_DIR` (por defecto, la carpeta del proyecto — igual que siempre). En producción, para que las salas y la agenda sobrevivan un redeploy, hace falta un disco persistente montado ahí (por ejemplo un [Render Disk](#) en `/data` con `DATA_DIR=/data`); sin eso, el próximo `git push /deploy` vuelve a dejar el `sessions.json` del repositorio.

3.2. Código de orador — habilitar a quien va a hablar sin saber su mail de antemano

`SPEAKER_EMAILS` (sección 8) sirve cuando de antemano sabés el mail de cada orador. En un evento con varias salas y un lineup que se arma sobre la marcha, eso no siempre es posible. Para esos casos, el panel `/admin/sessions` puede generar, sala por sala, un código temporal que habilita a transmitir sin agregar a nadie a ninguna lista:

1. En la tarjeta de la sala, apretá **🔑 Código de orador**. Aparece un código de 6 dígitos, válido durante 4 horas.
2. Dictáselo (de palabra, por WhatsApp, como sea) a quien va a dar la charla en esa sala.
3. Esa persona entra a `/operator/<id>` (el link de “operador” de la sala), inicia sesión con Google o GitHub —lo mismo que ya hace cualquier oyente para chatear— y carga el código en el cuadro **Habilitar como orador de esta sala**.
4. A partir de ahí puede transmitir el audio (sección 4.1) **solo en esa sala**: el mismo código no sirve para transmitir en ninguna otra, y expira solo a las 4 horas.

Si `SPEAKER_EMAILS` está vacío (el valor por defecto), cualquier cuenta logueada ya puede transmitir en cualquier sala sin necesitar código — el código de orador es la forma de acotar eso sin tener que armar la lista de mails de antemano.

4. Operación durante el evento

4.1 Guía paso a paso: armar una sala y ponerla en vivo, de punta a punta

Esta es la secuencia completa, en orden, desde que se crea la sala hasta que la audiencia ve los subtítulos. Es la misma que se sigue en el video de demo.

A. El organizador/admin crea la sala (una vez, antes del evento o del bloque de charlas)

1. Entra a `https://captions.miconferencia.org/admin/sessions` e inicia sesión con Google o GitHub. Con una cuenta de `ADMIN_EMAILS` administra todas las salas del evento; con cualquier otra cuenta puede crear y administrar su propia sala de prueba para ensayar estos mismos pasos sin pedirle nada a nadie (sección 3.1, punto 4).
2. Completa **Nueva sala:** id (aparece en la URL, ej. `sala-abasto`), nombre, ubicación, color, idioma de origen y a qué idiomas traducir. Aprieta **+ Crear sala**.
3. Opcional: carga la agenda de esa sala (título, orador, horario) y una nota de la fuente de audio prevista.
4. Cuando ya sabe quién va a hablar en esa sala, aprieta **🔑 Código de orador** y le pasa el código de 6 dígitos al orador (ver 3.2 arriba). Este paso se puede repetir charla a charla si cambia el orador.
5. Puede cerrar sesión en cualquier momento con el botón **Cerrar sesión** de la cabecera (por ejemplo para probar con otra cuenta de Google).

B. El orador conecta el audio de su charla, por uno de estos dos caminos:

- **Camino 1 — Micrófono conectado a la PC** (el caso típico: la notebook del orador o del operador de escenario, con la salida de la consola de sonido de la sala cableada a su entrada de línea, o directamente su propio micrófono):
 1. Abre `/operator/<id>` de su sala (se lo pasa el organizador junto con el código).
 2. Si el sistema no lo reconoce todavía como orador, ve el cuadro **Habilitar como orador de esta sala**: inicia sesión con Google o GitHub y carga el código de 6 dígitos (paso A.4). Si ya es orador habilitado (`SPEAKER_EMAILS` o sesión previa), no ve este cuadro y pasa directo al punto siguiente.
 3. En **Fuente de audio**, elige la entrada correcta del desplegable, o aprieta **🔍 Detectar micrófono**: prueba cada entrada disponible durante 2 segundos mientras el orador habla, y deja seleccionada la que tuvo señal.
 4. Aprieta **▶ Iniciar con micrófono / entrada**. El navegador pide permiso de micrófono la primera vez — aceptar. El medidor de nivel debajo se tiene que mover cuando habla.
 5. Abajo, en **Últimos subtítulos**, aparece en vivo lo que el sistema está transcribiendo — así el orador confirma que está funcionando antes de arrancar la charla en serio.
- **Camino 2 — Pestaña de streaming del navegador** (la charla llega por Zoom, Meet, YouTube en vivo, o cualquier pestaña con audio — no hay micrófono físico conectado a esa PC):
 1. Mismos pasos B.1 y B.2 de arriba (login + código si hace falta).
 2. Aprieta **▶ Capturar audio de una pestaña**. Chrome muestra el selector “Elegir qué compartir” — hay que elegir la pestaña correcta (no toda la pantalla) y **tildar “Compartir audio de la pestaña”** (si no se tilda, no hay señal).
 3. El sistema descarta el video y se queda solo con el audio de esa pestaña; el resto es igual que el camino 1 (medidor de nivel, preview de subtítulos).

4. Si el audio sale de un encoder de streaming (RTMP/SRT) y no de una pestaña del navegador, se puede evitar la notebook en la sala directamente con `node tools/ingest.js --session <id> --input rtmp://encoder/live/<id> --token $INGEST_TOKEN` corriendo en el servidor.

En ambos caminos: dejar esa pestaña abierta y la notebook sin suspender durante todo el bloque; si se corta la conexión, la consola reintenta sola. Al terminar, apretar **■ Detener**.

C. El oyente entra a ver los subtítulos, con todas las funciones disponibles (detalladas con capturas en 4.2 a 4.4):

1. Abre `https://captions.miconferencia.org/` (o el QR de la sala) y elige la sala y el idioma. Directo a una sala: `/s/<id>?lang=es`.
2. Ve los subtítulos en vivo, con **ORIG** para mostrar también la frase original y **A-/A+** para el tamaño de letra.
3. Si llega tarde, **⌂ → ¿Qué me perdí?** le da un resumen con IA de lo dicho hasta ese momento, en su idioma.
4. Puede cambiar de sala sin volver a la portada (**▼** junto al nombre de la sala), iniciar sesión con Google/GitHub para chatear, levantar la mano y pedir la palabra, activar **Alto contraste** o **Pausar para releer**, y exportar o mandarse por mail la transcripción de la charla.
5. Si inició sesión, puede cerrar sesión o cambiar de cuenta desde su tarjeta de cuenta en la cabecera.

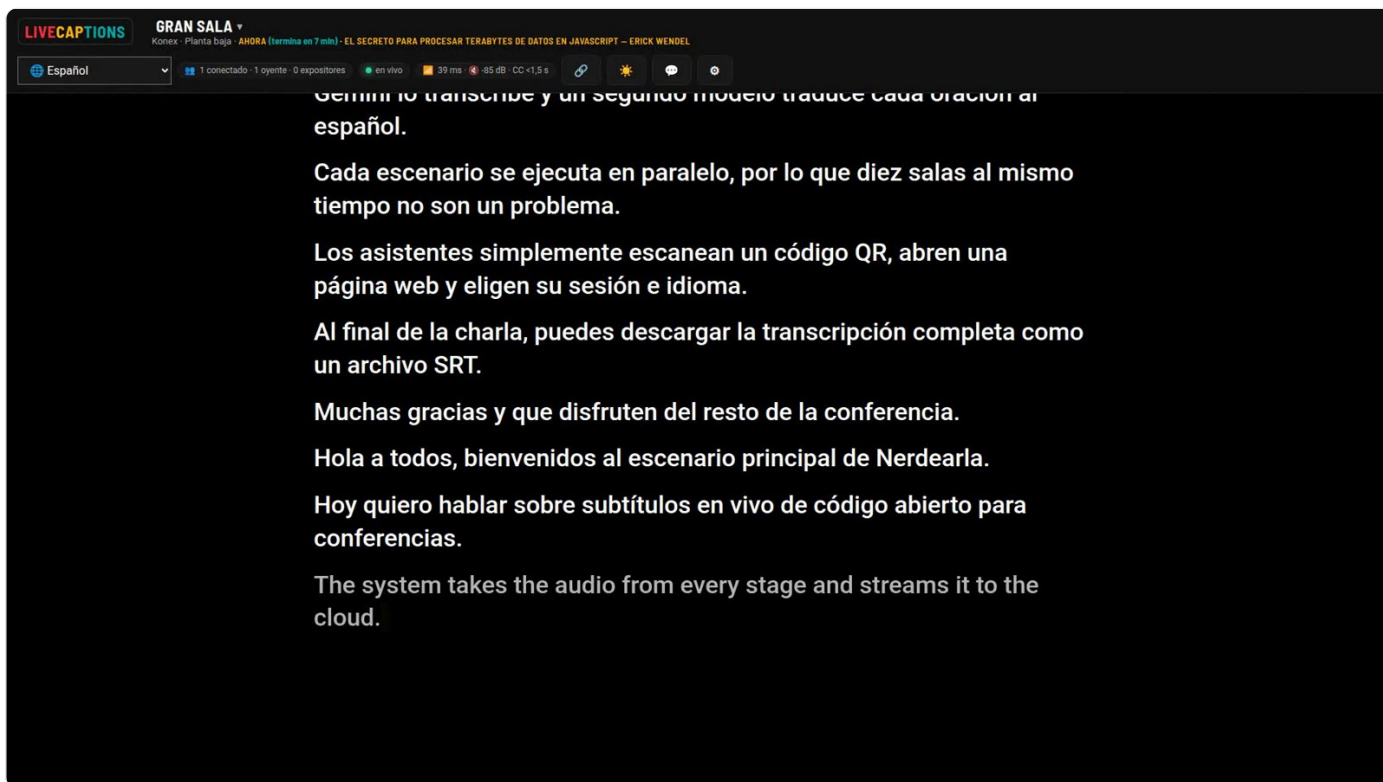
4.1.1 Antes de cada bloque de charlas (operador) — resumen rápido

1. En la notebook del escenario abrí `https://captions.miconferencia.org/operator/<id>`.
2. Conecté la salida de la consola de sonido a la entrada de audio de la notebook (line-in o interfaz USB). Elegí esa entrada en **Fuente de audio**.
3. Si el servidor exige `INGEST_TOKEN`, pegalo (queda guardado en el navegador); si en cambio el organizador te dio un **código de orador** (sección 3.2), inicié sesión y cargalo en el cuadro que aparece arriba de todo.
4. Apreté **Iniciar**. El medidor de nivel debe moverse cuando alguien habla. Si está en cero, revisé la fuente elegida y el volumen de la consola.
5. Dejé esa pestaña abierta y la notebook sin suspender durante todo el bloque.

Alternativas a la consola web:

- **Desde el streaming**: si el escenario ya sale por RTMP/SRT, en el servidor corrí `node tools/ingest.js --session main --input rtmp://encoder/live/main --token $INGEST_TOKEN` y no hace falta ninguna notebook en la sala.
- **Desde una pestaña**: para charlas remotas (Zoom, Meet, YouTube), el botón **Capturar audio de una pestaña** toma el audio de esa pestaña.

4.2 Qué ve la audiencia



Subtítulos en vivo con traducción y controles de idioma

- `https://captions.miconferencia.org/` lista los escenarios; cada tarjeta muestra si hay audio en vivo.
- `https://captions.miconferencia.org/s/main?lang=es` es la vista de subtítulos. Ese es el link para el **QR de la sala**.
- Botones: idioma, **ORIG** (muestra también la frase original debajo de la traducción), **A-/A+** tamaño de letra.
- Quien entra tarde ve las últimas frases de la charla, y con **⌂ → ¿Qué me perdí?** obtiene un resumen con IA de lo dicho hasta ahora, en su idioma.
- **Cambiar de sala sin volver a la portada**: click en el nombre de la sala (**▼**) en la cabecera. Se abre la lista de todas las salas, primero las que están **en vivo** (“3 charlas en simultáneo” si hay varias), cada una con su charla actual o siguiente, la cuenta regresiva (“termina en 19 min” / “empieza en 9 min”) y cuánta gente está conectada; un click cambia de sala manteniendo el idioma. La cabecera muestra la charla actual o la que sigue con la cuenta regresiva, y se actualiza sola con la agenda.
- **Tu cuenta**: click en tu nombre en la cabecera abre tu cuenta (nombre, mail, rol) con **Cerrar sesión** y **Cambiar nombre / rol**. Quien entra sin cuenta ve el badge *invitado · solo lectura*; clickearlo abre el inicio de sesión.
- **⌂ → Pausar para releer** congela la pantalla (los subtítulos siguen llegando por detrás) y **Volver al vivo** la retoma. **Alto contraste** activa el modo para baja visión / personas sordas: tipografía Atkinson Hyperlegible, letra grande, últimas tres líneas resaltadas. Se recuerda en ese dispositivo.

4.3 Pantalla de la sala y OBS

Agregá `&overlay=1` a la URL de la audiencia para obtener subtítulos grandes, centrados abajo, con fondo transparente:

- **OBS / vMix:** fuente *Browser* con `https://captions.miconferencia.org/s/main?lang=es&overlay=1&fs=44` , ancho 1920 × alto 1080. Los subtítulos quedan “quemados” en el stream.
- **Pantalla lateral:** un navegador a pantalla completa con la misma URL.

`fs=` fija el tamaño de letra en píxeles.

- **Pantalla de sala con QR:** `https://captions.miconferencia.org/s/main?lang=es&tv=1` muestra los subtítulos grandes y el QR de la sala abajo a la derecha, para el proyector o una TV lateral.
- **Tarjeta de acceso de la sala** (para imprimir en la puerta o proyectar entre charlas): `https://captions.miconferencia.org/s/main/qrcode?lang=es` — se abre con el botón **QR** de cada tarjeta de la portada o con  → **QR de la sala**. Trae el QR grande sobre el fondo y color de la sala, nombre y ubicación, evento y fecha, la charla actual o siguiente con cuenta regresiva, la agenda completa del día (la charla en vivo resaltada, las pasadas atenuadas) y los idiomas disponibles. **Imprimir** la saca en A4 en blanco; en pantalla se actualiza sola cada 30 s. El QR pelado, para otros usos: `/api/sessions/main/qrcode.svg` (usa `PUBLIC_URL` del `.env`).
- **vMix / OBS fuente de texto / carteles LED:** `/api/sessions/main/now.txt?lang=es` devuelve el último subtítulo como texto; `now.json` agrega estado (`live` , `deadAir`) y hora.

Los asistentes solo tienen que
escanear un código QR, abrir
una página web y elegir su
sesión e idioma.

Al finalizar la charla, puede
descargar la transcripción
completa como un archivo SRT.

Muchas gracias y disfruten del
resto de la conferencia.

Hello everyone, welcome to the
main stage of Nerdearla.

Today I want to talk about open
source live



Tarjeta de acceso de la sala con QR y agenda del día

4.4 Panel de producción

PANEL DE PRODUCCIÓN

Estado de cada sesión, latencia y errores · se actualiza cada 2 s

● motor: mock

SESIÓN	AUDIO	MOTOR	ÚLTIMO SUBTÍTULO	TRADUCCIÓN	AUDIENCIA	SEGMENTOS	ERRORES	LINKS
Gran sala gran-sala · Konex · Planta baja · auto → es, en	demo · 0.0 s 26 s recibidos · -90 dBFS · SIN SEÑAL	conectado mock	parcial 0.3 s final 1.4 s	251 ms 6 ok	2	6	0	operador · audiencia · SRT · pantalla+QR · QR · now.txt
Auditorio auditorio · Konex · Auditorio · auto → es, en	sin fuente 0 s recibidos ·	inactivo	parcial — final —	— 0 ok	0	0	0	operador · audiencia · SRT · pantalla+QR · QR · now.txt
Sala Abasto sala-abasto · Konex · Abasto · auto → es, en	sin fuente 0 s recibidos ·	inactivo	parcial — final —	— 0 ok	0	0	0	operador · audiencia · SRT · pantalla+QR · QR · now.txt
Container gris container-gris · Patio del Konex · auto → es, en	sin fuente 0 s recibidos ·	inactivo	parcial — final —	— 0 ok	0	0	0	operador · audiencia · SRT · pantalla+QR · QR · now.txt

Audio = hace cuánto llegó el último paquete de audio del escenario. *Último subtítulo* = hace cuánto se emitió el último parcial / final. *Traducción* = tiempo promedio de traducción por segmento. *SIN SEÑAL* = llega audio pero por debajo de -50 dBFS durante 20 s (cable/console muteados). Métricas Prometheus en [/metrics](#).

Panel de producción /admin: audio, motor, latencia y audiencia por sesión

`https://captions.miconferencia.org/admin` muestra, por sesión:

- **Audio**: hace cuánto llegó el último paquete del escenario. Si pasa de unos segundos en rojo, el operador perdió la captura.
- **Motor**: si la conexión con Gemini está activa.
- **Último subtítulo**: hace cuánto salió el último parcial/final. Si hay audio pero no salen subtítulos, mirá los errores.
- **Traducción**: latencia promedio por frase.
- **Audiencia**: cuántas personas están conectadas a esa sesión.
- **Errores** y links rápidos al operador, la vista de audiencia, la pantalla con QR, el QR solo, `now.txt` y el SRT.
- **SIN SEÑAL** en la columna Audio: llega audio pero por debajo de -50 dBFS durante 20 s. Casi siempre es un cable desconectado, la consola muteada o la entrada equivocada.
- Para Grafana u otro monitoreo, `GET /metrics` expone todo en formato Prometheus.

4.5 Al terminar cada charla

Bajá la transcripción para publicarla con el video:

- `https://captions.miconferencia.org/api/sessions/main/transcript.srt?lang=es`
- Formatos: `.srt`, `.vtt`, `.txt`, `.json`. Sin `?lang=` devuelve el idioma original.

Los archivos también quedan en la carpeta `transcripts/` del servidor (`<id>.jsonl`), un archivo por sesión que acumula toda la jornada.

5. Problemas frecuentes

Síntoma	Causa probable	Qué hacer
El operador no puede iniciar el micrófono	La página no es HTTPS	Usar el dominio con TLS, no la IP.
“no se pudo conectar {¿token?}” en el operador	<code>INGEST_TOKEN</code> incorrecto	Copiar el token del <code>.env</code> del servidor.
Medidor en cero	Entrada de audio equivocada o muteada	Cambiar Fuente de audio ; revisar la consola de sonido.
Subtítulos aparecen pero la traducción tarda o falta	Cuota de la API agotada (nivel gratuito)	Activar facturación en Google Cloud; el sistema rota entre modelos alternativos mientras tanto.
Se corta cada ~10 minutos	Límite de sesión del proveedor	El servidor rota la sesión solo; si ves cortes, subí <code>SESSION_ROTATE_MS</code> un poco por debajo de 10 min.
Idioma detectado incorrecto en la primera frase	Detección automática con poco audio	Se corrige solo en las frases siguientes; si el escenario es monolingüe, fijá <code>sourceLang</code> .
Nombres propios mal transcritos	Falta vocabulario	Agregarlos a <code>vocabulary</code> en <code>sessions.json</code> .

6. Costos y escala

- Cada sesión abre una conexión de transcripción en streaming y hace una llamada de traducción por frase (más algunas por parciales largos). Con

- `flash-lite` el costo es de pocos dólares por hora de escenario; ver <https://ai.google.dev/pricing>.
- El servidor es liviano: el trabajo pesado lo hace la API. Diez escenarios en paralelo corren en un contenedor de 1 GB.
- Para reducir costo: `TRANSLATE_PARTIALS=0`, o `TRANSLATOR=ollama` con Gemma en una máquina local.

7. Modo demo y pruebas

- `npm run mock` levanta el servidor sin API key con una charla simulada: sirve para probar la interfaz y los QR.
- `./tools/demo-parallel.sh` mete las charlas de ejemplo de `demo/` en cinco sesiones a la vez.
- `node tools/ingest.js --session main --input demo/talk_en.mp3` reproduce una charla en inglés a velocidad real contra el servidor; en `/s/main?lang=es` se ve el resultado.

8. Versión 2: cuentas, chat y mail

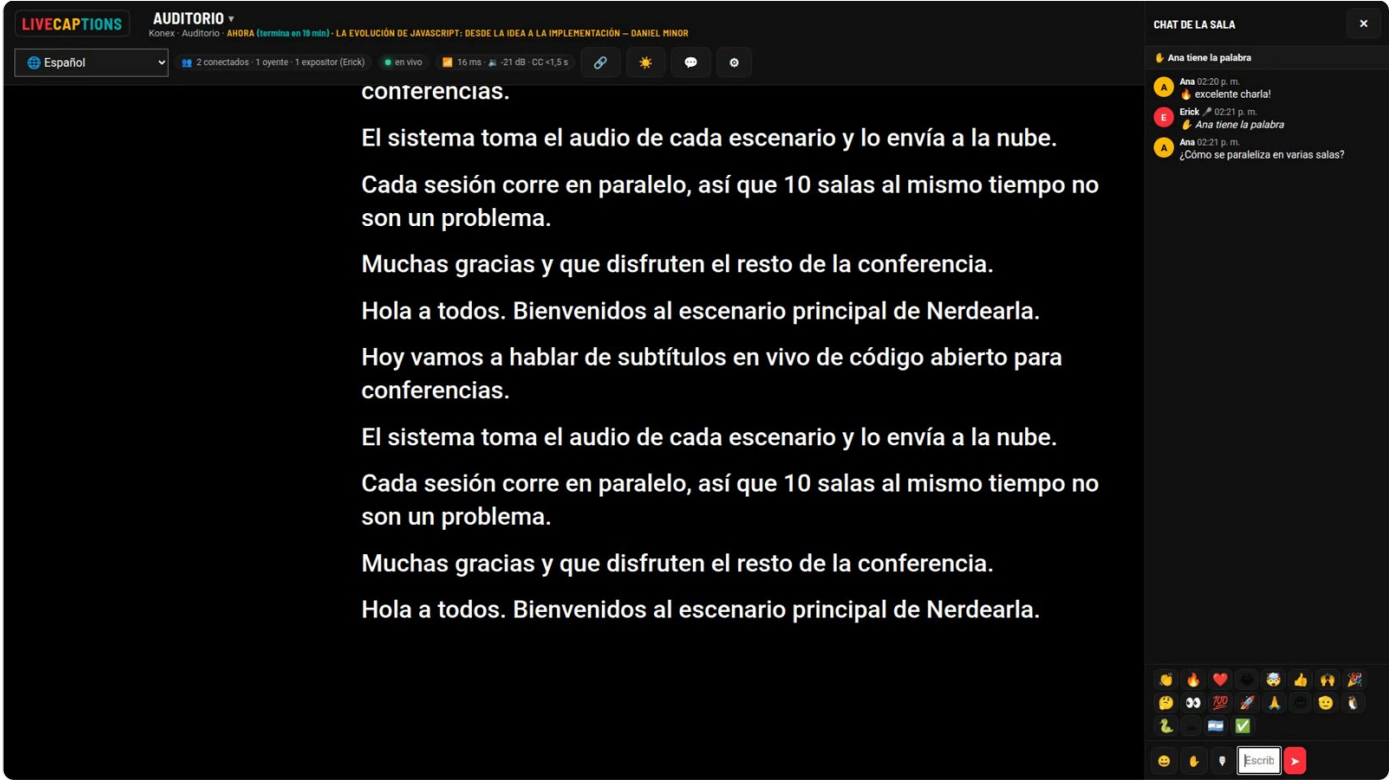
Todo lo de esta sección es opcional: sin configurar nada, la sala funciona como en la versión 1 (entrada con nombre, sin cuenta).

Inicio de sesión con Google o GitHub

Para GitHub: creá una *OAuth App* en GitHub → Settings → Developer settings, con callback `https://captions.tu-dominio.org/api/auth/github/callback`, y poné `GITHUB_CLIENT_ID` y `GITHUB_CLIENT_SECRET` en `.env`. Para Google:

- En <https://console.cloud.google.com/apis/credentials> creá un **ID de cliente OAuth** de tipo *Aplicación web*. En *Orígenes autorizados de JavaScript* agregá `https://captions.tu-dominio.org` (y `http://localhost:8080` para probar en tu PC).
- Poné el ID en `.env` como `GOOGLE_CLIENT_ID=...` y reiniciá.
- En la pantalla de bienvenida de la sala aparecen *Iniciar sesión con Google* y *Entrar con GitHub* (también en ⚙ → Yo). El nombre y la foto de la cuenta se muestran en la cabecera, en la lista de conectados y en el chat. Cerrar sesión o cambiar de cuenta: en la vista de audiencia, tu tarjeta de cuenta en la cabecera; en `/admin/sessions`, el botón **Cerrar sesión** de la cabecera; en `/operator/<id>`, **No soy yo, cambiar de cuenta** junto al cuadro de código de orador. Para probar en tu PC, agregá `http://localhost:8080/api/auth/github/callback` como segunda callback en la OAuth App de GitHub.
- `GUEST_ACCESS` define qué puede hacer quien entra sin cuenta cuando hay login configurado: `limited` (por defecto: sólo lee los subtítulos, con un nombre validado; sin chat, mano, expositor ni mail — ve un aviso y un botón para iniciar sesión), `full` (igual que una cuenta) u `off` (obligatorio iniciar sesión; equivale a `ALLOW_GUESTS=0`). `SPEAKER_EMAILS=ana@conf.org,juan@conf.org` limita quién puede transmitir como expositor de antemano; vacío = cualquiera con sesión puede (usá entonces el [código de orador](#) de la sección 3.2 si igual querés acotarlo sala por sala, sin armar la lista de mails). `ADMIN_EMAILS=vos@gmail.com` habilita el [panel de salas](#) (sección 3.1) para esos mails; vacío = panel deshabilitado. `ALLOWED_DOMAINS=tu-dominio.org` limita quién puede entrar.

Chat de la sala



Chat de la sala con mano alzada y palabra concedida

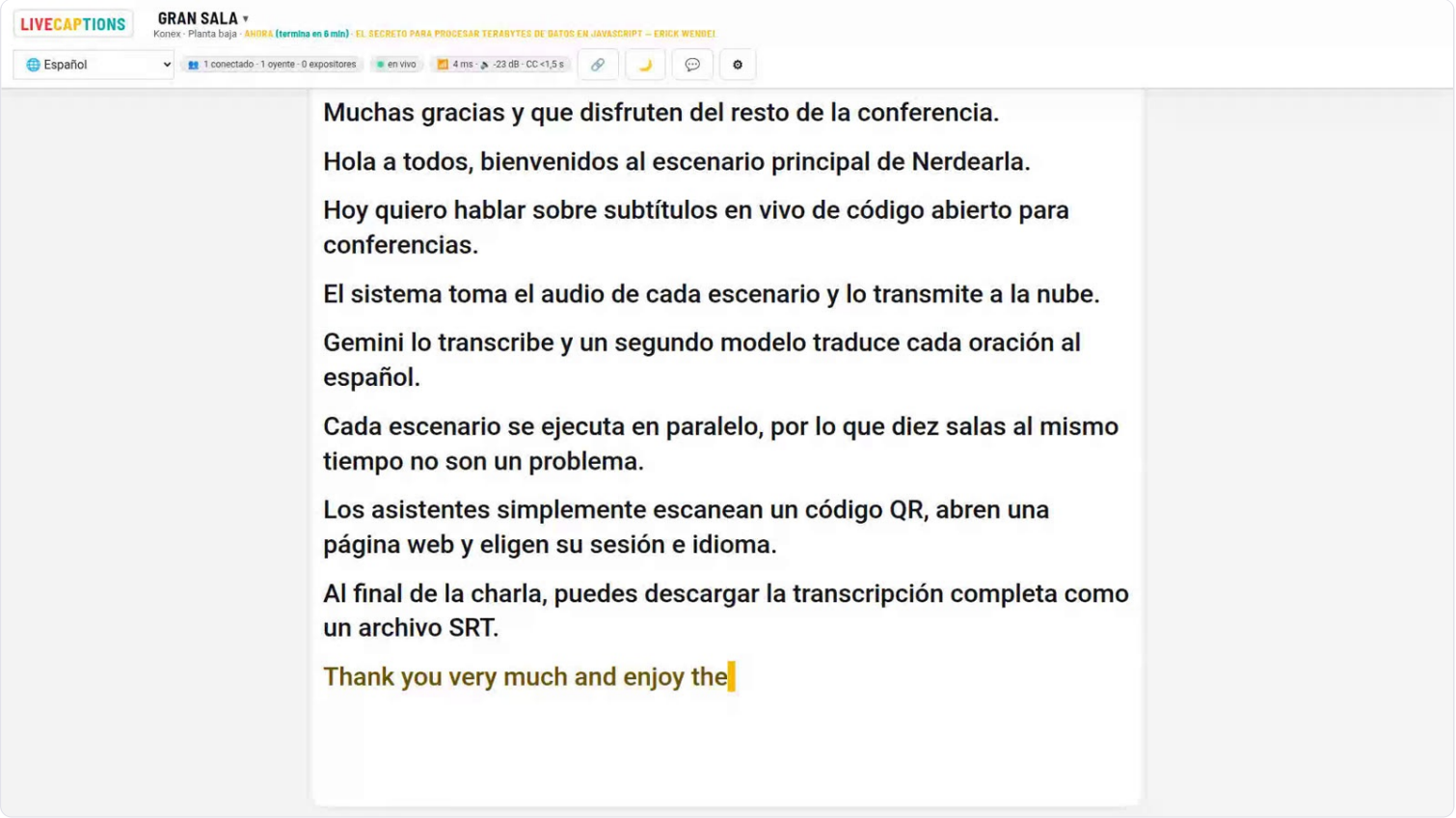
Botón en la cabecera. Tiene emojis, audios (mantener apretado , hasta 20 s) y **levantar la mano** : el expositor ve las manos levantadas en el chat y con *dar la palabra* habilita a esa persona; mientras tanto nadie más puede escribir (ven “Esperá: X tiene la palabra”) hasta que el expositor apreta *Cerrar palabra*. Los mensajes se guardan con la transcripción (`transcripts/<id>.jsonl`) y los últimos 50 se muestran a quien entra tarde. Los expositores aparecen con .

“¿Qué me perdí?” y modo día



Resumen con IA “¿Qué me perdí?” y el modo claro

El botón  → **¿Qué me perdí?** (sección 4.2) y el interruptor de tema día/noche de la cabecera (



) funcionan en cualquier sala, con o sin cuenta.

Enviar la transcripción por mail

En  → *Exportar* → escribí el mail destino (si iniciaste sesión, se completa con el tuyo), elegí “desde / hasta” y formato, y **Enviar por mail**. El servidor necesita un proveedor:

- **SMTP** (`SMTP_URL`): con Gmail usá una *contraseña de aplicación* (Cuenta de Google → Seguridad → Verificación en dos pasos → Contraseñas de aplicaciones): `SMTP_URL=smtps://tu-mail%40gmail.com:xxxx-xxxx-xxxx-xxxx@smtp.gmail.com:465`
- **Resend** (`RESEND_API_KEY`): sin SMTP, `https://resend.com` (gratis hasta 3.000 mails/mes) con tu dominio verificado.

`MAIL_FROM` define el remitente y `PUBLIC_URL` el link a la sala que va en el mail.